

BORDER OPTICS — POORI RESEARCH SAMJHO

PDF 16 of 22 — Robustness and Advanced Stats: Multiple-Testing and Spatial Correction

holm_correction.py, spatial_moran_and_h3_correction.py

Sakshi ke liye, Sakshi ki apni research par. Is poore project mein humne ab tak kai alag-alag statistical tests dekhe hain — H1 ka Wilcoxon test, H3 ka Spearman correlation, H4 ka DiD, PDF 14 ka wild cluster bootstrap, PDF 15 ke placebo tests. Har test apne aap mein theek se design kiya gaya tha. Lekin ek naya, subtle sawaal hai jo tab uthta hai jab ek hi study mein bahut saare tests ek saath report kiye jaate hain: agar 8 tests chalaye jaayein aur har ek mein sirf 5% galti ka chance ho, toh poori study mein kam-se-kam ek false positive milne ka combined chance 5% se kaafi zyada ho jaata hai — sirf random luck ki wajah se. Is PDF ka Part A, `holm_correction.py`, exactly isi problem ko theek karta hai.

Part B, `spatial_moran_and_h3_correction.py`, ek bilkul alag lekin isi family ka sawaal poochta hai: kya 251 core-sample villages sach mein 251 independent, ek-doesre se bekhabar observations hain — ya kya paas-paas ke villages ek-doesre se milte-julte outcomes dikhate hain, sirf apni geography ki wajah se? Agar haan, toh H3 ke Spearman correlation ka p-value bhi zyada optimistic nikal raha hoga, jitna hona chahiye usse. Dono scripts is project ki sabse mature, sabse self-critical stage ko represent karte hain — jahan researcher khud apne significant results par doosri, sakhter nazar daal rahi hain, sirf ek baar significant mil jaane se santusht nahi ho rahi.

Shuru Karne Se Pehle — 'Multiple Testing Problem' Kya Hai, Aur Ye Kyun Zaroori Hai

Is study mein ab tak alag-alag PDFs mein alag-alag hypotheses test hui hain — H1 (kya treated villages mein NDBI/lights before-se-after badhta hai), H3 (kya ye badhaav border ke paas zyada hota hai), aur inhe do windows mein — full-year aur summer-matched — bhi test kiya gaya hai. Jab in sabko jodo, toh ek hi paper mein total 8 alag p-values report ho rahe hain (§4.9 ki table): H1 ka NDBI-change aur lights-change, dono windows mein (4 tests), aur H3 ka border-proximity, NDBI aur lights ke liye, dono windows mein (4 tests).

Ek single test ke liye, ' $p < 0.05$ ' ka matlab hai: agar sach mein koi effect nahi hota (null hypothesis sach hoti), tab bhi sirf random chance se itna extreme result milne ka chance 5% ya usse kam hai. Lekin jab 8 alag tests ek saath chalaye jaayein — sabhi ek jaisi 5%-threshold ke saath — toh sabhi 8 mein se kam-se-kam ek test ka galti se 'significant' dikh jaana, sirf luck se, kaafi zyada likely ho jaata hai. Agar har test independent hota (jo yahan poori tarah sach nahi hai, lekin intuition ke liye), toh kam-se-kam-ek-galti ka combined chance roughly $1 - (0.95)^8$, yaani lagbhag 34% ban jaata hai — 5% nahi, balki 34%. Yehi 'multiple testing problem' hai, aur Holm-Bonferroni correction (Part A) exactly isi ko theek karne ke liye bana hai.

EXAMPLE — Lottery Ticket Wali Analogy

Socho ek insaan ek lottery ticket khareedta hai — jeetne ka chance bahut kam hota hai, maano 5%. Lekin agar wahi insaan 8 alag-alag lotteries mein ticket khareed le, toh kam-se-kam ek lottery jeetne ka overall chance 5% se kaafi zyada ban jaata hai — sirf isliye kyunki usne bahut saari koshishen ki hain, na ki kisi ek ticket ke 'special' hone ki wajah se. Bilkul waisa hi statistics mein hota hai: jitne zyada tests chalo, utna zyada chance ki koi ek test, sirf random noise ki wajah se, 'significant' dikh jaaye — bina kisi asli effect ke. Holm-Bonferroni correction har p-value ko is baat ke hisaab se 'mehnga' bana deta hai ki kitne total tests chalaye gaye the — jitne zyada tests, utna sakht standard har ek ko paar karna padta hai significant kehlaane ke liye.

Spatial autocorrelation ka sawaal thoda alag hai, lekin isi 'kya humara result sach mein jitna dikh raha hai utna reliable hai' family ka hai. H3 ka test yeh maanta hai ki 251 core-sample villages independent observations hain — matlab ek village ka outcome doosre village ke outcome se poori tarah bekhabar hai. Lekin agar geography ki wajah se paas-paas ke villages mein-se-mein similar outcomes dikhte hain (jaise ek hi micro-climate, ek hi local economy, ek hi infrastructure-corridor), toh asli, effective sample size 251 se kam hai — aur isse har statistical test ka p-value, jitna hona chahiye usse zyada optimistic (zyada significant dikhne wala) ho jaata hai. Part B isi ko test aur theek karta hai.

Part A — holm_correction.py

Docstring Aur Imports

```
"""
Holm-Bonferroni correction across the 8 headline p-values (Section 4.9).
Note: the lights before/after Wilcoxon test is labeled H1 here, not H3 -
it's the same design as H1's NDBI test, no border distance involved.
```

```
Run from repo root: python src/analysis/holm_correction.py
"""
```

```
import pandas as pd
from scipy import stats
from statsmodels.stats.multitest import multipletests
```

Purpose: Docstring seedha bata deta hai ki ye file kya karti hai: paper ke §4.9 mein report hone wale 8 headline p-values par Holm-Bonferroni correction lagana. Ek chhota lekin zaroori clarification bhi hai — lights ka before/after test H3 (border-proximity) nahi, balki H1 (treated-villages-ka-apna change) ke under aata hai, kyunki iska formula/design NDBI ke H1 test jaisa hi hai, border-distance se kuch lena-dena nahi. Ye naming-clarity is baat ka proof hai ki paper likhte waqt har test ko sahi jagah, sahi naam ke saath rakhna khud ek disciplined kaam hai. Syntax: import pandas as pd — dataframes ke liye, is poore series mein bahut baar dekha. from scipy import stats — Wilcoxon aur Spearman jaise statistical tests ke liye (PDF 11 aur PDF 12 mein pehli baar dekhe the). from statsmodels.stats.multitest import multipletests — ye is file ki sabse important, naya import hai — ek specific function jo multiple-testing corrections (Bonferroni, Holm, aur kai doosre methods) implement karta hai, taaki hume ye arithmetic khud se likhne ki zaroorat na pade.

RESULT_FILES Aur DISTANCES_PATH

```
RESULT_FILES = {
    "full-year": "data/processed/border_optics_village_results_analyzed.csv",
    "summer-matched":
    "data/processed/border_optics_village_results_summer_analyzed.csv",
}
DISTANCES_PATH = "data/processed/border_optics_master_villages_with_distance.csv"
```

Purpose: Do windows (full-year, summer-matched) ke analyzed-results files ka path dictionary mein store, aur ek alag file jo har village ka border-se-distance rakhti hai (H3 ke liye zaroori) — ye wahi distances file hai jo PDF 13 mein compute_border_distance.py se banti hai. Syntax: Dictionary literal, string keys ke saath — PDF ki is series mein pehle bhi kai baar dekha gaya pattern, do file-paths ko unke window-naam se jodna.

wilcoxon_ndbi_lights() — H1 Ke Dono Outcomes Ke Liye Ek Function

```
def wilcoxon_ndbi_lights(path):
    df = pd.read_csv(path)
    df["ndbi_change"] = df["ndbi_after"] - df["ndbi_before"]
    df["lights_change"] = df["lights_after"] - df["lights_before"]
    core = df[df["is_core_sample"] == True].copy()

    valid_ndbi = core.dropna(subset=["ndbi_before", "ndbi_after"])
    _, p_ndbi = stats.wilcoxon(valid_ndbi["ndbi_after"], valid_ndbi["ndbi_before"],
    alternative="greater")

    valid_lights = core.dropna(subset=["lights_before", "lights_after"])
    _, p_lights = stats.wilcoxon(valid_lights["lights_after"], valid_lights["lights_before"],
    alternative="greater")

    return p_ndbi, p_lights
```

Purpose: Ye function ek window (full-year ya summer) ke analyzed-results file ko padhta hai, NDBI aur lights ke change columns banata hai (after minus before — bilkul PDF 11 ke analyze_results.py jaisa), sirf core-sample villages rakhta hai, aur dono outcomes ke liye alag-alag Wilcoxon signed-rank test chalata hai — jo exactly wahi test hai jo H1 mein PDF 11 mein pehli baar dekha tha: 'kya treated villages ka after-value, before-value se significantly zyada hai.' alternative="greater" batata hai ki ye ek one-sided test hai — hum sirf 'increase hua kya' poochh rahe hain, 'kisi bhi taraf farak hua kya' nahi. Syntax: df["a"] - df["b"] — do columns ka element-wise subtraction, poore column par ek saath. .dropna(subset=[...]) — un rows ko hata dena jinme specified columns mein missing value hai. stats.wilcoxon(x, y, alternative="greater") — ek underscore (_) ka use karke pehla return-value (test statistic) discard kiya jaata hai, kyunki hume sirf p-value chahiye — Python mein ye ek common convention hai, jab ek function multiple values return kare lekin sabki zaroorat na ho.

h3_border_proximity() — H3 Ke Dono Outcomes Ke Liye

```
def h3_border_proximity(path, distances):
    results = pd.read_csv(path)
    results["ndbi_change"] = results["ndbi_after"] - results["ndbi_before"]
    results["lights_change"] = results["lights_after"] - results["lights_before"]
    merged = results.merge(distances[["village_id", "distance_to_border_km"]],
    on="village_id", how="inner")
    merged = merged[merged["is_core_sample"] == True]

    out = {}
    for metric in ["ndbi_change", "lights_change"]:
        valid = merged.dropna(subset=[metric, "distance_to_border_km"])
        _, p = stats.spearmanr(valid["distance_to_border_km"], valid[metric])
```

```
out[metric] = p
return out
```

Purpose: Bilkul upar wale function jaisa hi structure, lekin H3 ke liye — border-se-distance ka column results file se join karna (village_id ke through), phir dono outcomes (ndbi_change, lights_change) ke liye Spearman correlation nikaalna — exactly wahi test jo PDF 13 mein pehli baar dekha tha, jismein distance aur change ke beech ek monotonic relationship dhoondha jaata hai. Result ek dictionary mein store hota hai, key = metric ka naam. Syntax: .merge(df2, on="village_id", how="inner") — pandas ka join, PDF 15 mein bhi dekha. Dictionary comprehension jaisa loop — for metric in ["a", "b"]: out[metric] = ... — ek list ke har item ke liye kuch calculate karke dictionary mein store karna. stats.spearmanr(x, y) — do arrays ka Spearman rank correlation aur uska p-value ek saath return karta hai; yahan bhi pehla return-value (correlation coefficient khud) discard kiya gaya hai, sirf p-value chahiye.

main() — Saare 8 Tests Ikattha Karna, Fir Holm Correction Lagana

```
def main():
    distances = pd.read_csv(DISTANCES_PATH)

    tests = {}
    for window, path in RESULT_FILES.items():
        p_ndbi, p_lights = wilcoxon_ndbi_lights(path)
        tests[f"H1_ndbi_change_{window}"] = p_ndbi
        tests[f"H1_lights_change_{window}"] = p_lights

    h3 = h3_border_proximity(path, distances)
    tests[f"H3_border_proximity_ndbi_{window}"] = h3["ndbi_change"]
    tests[f"H3_border_proximity_lights_{window}"] = h3["lights_change"]
```

Purpose: Ek loop dono windows (full-year, summer-matched) ke through jaata hai. Har window ke liye, H1 ke dono outcomes ka p-value aur H3 ke dono outcomes ka p-value nikaal ke, ek single 'tests' dictionary mein daal deta hai, jiski key ek descriptive string hai jaise 'H1_ndbi_change_full-year'. Iske baad, loop khatam hone par, is dictionary mein poore 8 tests ke naam aur unke raw p-values maujood hain — exactly wahi 8 jo paper ke §4.9 table mein report hote hain. Syntax: for window, path in RESULT_FILES.items(): — dictionary ke har key-value pair ke through loop, dono ek saath unpack ho rahe hain. f"H1_ndbi_change_{window}" — f-string ke saath dynamically ek naam banana, jismein window ki actual value (jaise 'full-year') seedha string mein daal di jaati hai.

```
names = list(tests.keys())
raw_p = [tests[n] for n in names]
reject, adj_p, _, _ = multipletests(raw_p, alpha=0.05, method="holm")
```

Purpose: Dictionary se do parallel lists banayi jaati hain — names (sabhi 8 test-naam, ek fixed order mein) aur raw_p (unke corresponding raw p-values, same order mein). Fir `multipletests()` function ko poori raw_p list, `alpha=0.05`, aur `method="holm"` ke saath call kiya jaata hai — yehi ek line poore Holm-Bonferroni correction ka kaam karti hai: saare 8 p-values ko dekhkar, unke rank ke hisaab se adjust karke, batati hai kaunsa test 0.05 ke sakht standard par bhi significant reh jaata hai. Syntax: `list(dict.keys())` — dictionary ki saari keys ko ek list mein badalna, order preserve rehta hai (Python 3.7+ mein dictionaries insertion-order yaad rakhti hain). List comprehension `[tests[n] for n in names]` — names list ke har item ke liye, tests dictionary se uski value nikaalkar ek nayi list banana; ye zaroori hai taaki raw_p aur names dono ek hi order mein rahein. Function ka multiple return-values ko ek saath unpack karna: `reject, adj_p, _, _ = multipletests(...)` — `multipletests()` 4 values return karta hai (reject-flags, adjusted-p-values, aur do aur jo yahan zaroori nahi, isliye underscore se discard kiye gaye).

EXAMPLE — `multipletests()` Ke Andar Kya Ho Raha Hai — Holm Step-Down Procedure

Holm-Bonferroni ka tareeka simple hai lekin thoda counter-intuitive: sabhi raw p-values ko sabse chhote se sabse bade order mein sort karo. Sabse chhote p-value ko poore tests ki ginti (yahan 8) se multiply karo. Doosre sabse chhote ko $(8-1=7)$ se, teesre ko $(8-2=6)$ se, aur aise hi aage — jitna neeche jaao, multiplier chhota hota jaata hai. Ek zaroori rule bhi hai: adjusted p-value kabhi apne se pehle wale (chhote raw-p wale) adjusted p-value se kam nahi ho sakta — agar aisa hoga, toh usse pehle wale ke barabar bana diya jaata hai (isse 'monotonicity enforcement' kehte hain, taaki ek chhota-sa raw p-value wala test bhi bade raw-p wale test se kam adjusted-p na dikhaye, jo logically confusing hota). Aakhri mein, har adjusted value ko 1.0 par cap kiya jaata hai, kyunki koi bhi p-value 1.0 se zyada nahi ho sakta.

```
print(f'{"test":32s} {"raw p":>14s} {"holm-adjusted p":>18s} {"significant?"}')
print("-" * 82)
results_out = []
for name, raw, adj, rej in zip(names, raw_p, adj_p, reject):
    print(f'{"name":32s} {"raw":14.6e} {"adj":18.6e} {"YES" if rej else "no"}')
    results_out.append({"test": name, "raw_p": raw, "holm_adjusted_p": adj,
"significant_after_holm": bool(rej)})

out_df = pd.DataFrame(results_out)
out_path = "outputs/holm_correction_results.csv"
out_df.to_csv(out_path, index=False)
print(f"\nSaved: {out_path}")
```

Purpose: Ek formatted table console par print hoti hai — har row ek test, uska raw p, uska Holm-adjusted p, aur ek YES/no verdict. Saath-saath, har row ek dictionary ki tarah results_out list mein bhi jodi jaati hai. Loop khatam hone par, ye list ek dataframe mein badal ke ek CSV file mein save ho jaati hai — yehi wo holm_correction_results.csv hai jo hum abhi Final State mein padhenge. Syntax:

`zip(names, raw_p, adj_p, reject)` — 4 alag lists ko ek saath, position-by-position jodna, taaki ek single loop mein chaaron corresponding values milein. `f"{name:32s}"` — string formatting, name ko 32 characters ki width mein left-aligned print karna, taaki table ke columns seedhe mein lain (align) ho jaayein. `f"{raw:14.6e}"` — scientific notation mein print karna (jaise 2.644425e-06), 6 decimal digits ke saath, 14 characters ki total width mein. `bool(rej)` — numpy ka boolean type ko Python ke plain bool mein badalna, taaki JSON/CSV mein sahi tarah save ho. list of dicts se `pd.DataFrame(...)` banana — pandas ka common pattern, jab structured records ek list mein collect kiye gaye hote hain.

History — `holm_correction.py` Ke Saath Kya-Kya Hua

EXAMPLE — Entry 12/13 — Ye Script Pehli Baar Kab Likhi Gayi

`holm_correction.py` aur H3 ka Spearman-based border-proximity test dono Development Log ki Entry 12-13 ke around likhe gaye the, jab control-group design ka poora rationale set ho raha tha aur multi-year/buffer-sensitivity extraction scripts likhe ja rahe the. Ye is baat ka proof hai ki multiple-testing correction is study mein koi baad ka afterthought nahi thi, balki shuru se hi, jaise-jaise naye tests jodte gaye, unke against ek combined, corrected view rakhne ki koshish ki gayi.

EXAMPLE — Entry 14 — Output File Kabhi Committed Hi Nahi Hui Thi (Aur Ek Zaroori Wording Fix)

Development Log Entry 14 mein ek chhoti lekin zaroori bug pakdi gayi: `holm_correction.py` ka apna output (jo abhi hum `outputs/holm_correction_results.csv` mein dekh rahe hain) kabhi git repository mein commit hi nahi hui thi — script chal rahi thi, sahi results bhi bana rahi thi, lekin wo output-file khud version-control se bahar reh gayi thi, jaise ki wo exist hi na karti ho kisi aur ke liye jo repository check kare. Ise Entry 14 mein hi `restore/commit` kiya gaya. Isi entry mein ek doosra, conceptually zyada zaroori fix bhi hua: paper mein Holm correction ke baare mein jo wording thi, 'validates' jaisa strong word use kar rahi thi — jo galat impression deta hai, kyunki koi statistical correction kisi result ko 'validate' nahi karta, sirf false-positive rate ko control karta hai. Wording ko softened karke 'controls the false-positive rate' jaisa accurate phrasing diya gaya — ek chhota lekin zaroori honesty-fix, jo statistics ki language ko zyada precise banata hai.

EXAMPLE — Entry 16 — 'Blanket Description Overreach': Sabhi 6 Non-Significant Results Ek Jaisa Nahi The

Entry 16 mein ek zyada substantial prose-bug pakdi gayi, aur ye seedha upar jo Holm step-down procedure humne dekha, usi se related hai. Paper ke §4.9 mein likha tha ki jo 6 tests Holm correction ke baad significant nahi rahe, unke sabhi ke adjusted p-values 'exactly 1.000' tak pahunch gaye the. Ye galat tha — actual numbers dekho toh, `H1_lights_change_full-year` ka raw p sirf 0.050075 tha (rank 3, poore procedure mein), jo multiply hone ke baad (6 se) 0.300450 banta hai — 1.000 nahi, balki 0.300. Baaki paanch non-significant tests sach mein 1.000 tak cap ho gaye the, lekin ye ek

nahi hua. Ye bilkul PDF 15 ke us honesty-lesson jaisa hai — jahan ek script ka apna bare-threshold verdict ('PASSES') पूरी कहानी नहीं बताता — सिर्फ यहाँ फ़रक़ है कि वो ग़लती खुद कोड में नहीं, बल्कि पेपर की प्रोसे में थी, एक incorrect 'blanket' (sab-par-ek-jaisa) statement की तरह. Fix सिर्फ़ wording था — कोई नंबर नहीं बदला, सिर्फ़ ये सही तरह बताया गया कि 6 non-significant results में से एक genuinely close (0.300) था, बाकी पाँच बिल्कुल clean 1.000 थे.

EXAMPLE — Entry 18 — Distance Formula Fix Se Numbers Thode Shift Hue, Verdict Wahi Raha

Development Log Entry 18 में `compute_border_distance.py` में एक geometry-level bug fix हुआ — UTM Zone 44N projection use करके distance निकााली जा रही थी, जो North-East India के बड़े area (जहाँ multiple UTM zones cross होती हैं, जैसे Arunachal Pradesh) के लिये थोड़ा inaccurate हो सकती है; इसे geodesic (Earth की curvature को directly account करने वाला) distance-calculation से replace किया गया, जिसमें quantified bias Arunachal Pradesh में लगभग 3% तक थी. Is fix के बाद `holm_correction.py` फिर से चलायी गयी, और Entry 18 खुद ये exact numbers quote करता है: 'full-year NDBI-proximity: rho 0.043→0.049, p 0.497→0.443; full-year lights-proximity Holm-adjusted p: $2.24\times10^{-4}\rightarrow3.83\times10^{-4}$ ' — लेकिन, जैसा entry खुद जोर देकर लिखता है, 'not one test's significant/non-significant status changed.' ये is study का एक अच्छा example है कि हर underlying data-quality fix को rerun करके verify करना ज़रूरी है, चाहे expectation ये हो कि numbers थोड़े ही हिलेंगे — और यहाँ भी वही expectation सही निकली.

Final State — `holm_correction.py` Ka Current Verdict, पूरी Worked-Numeric Walkthrough

`outputs/holm_correction_results.csv` से सीधे लिये गये current 8 numbers, raw p-value के हिसाब से ascending order में sort करके — क्योंकि Holm procedure खुद इसी order पर काम करता है:

Rank	Test	Raw p	Multiplier	Holm-adjust ed p	Significant?
1	H3 border-p rox., NDBI, summer	2.6444e-06	×8	2.1155e-05	YES
2	H3 border-p rox., lights, full-year	5.4767e-05	×7	3.8337e-04	YES
3	H1 lights-ch ange, full-year	0.050075	×6	0.300450	no

4	H3 border-prox., NDBI, full-year	0.443353	×5	1.0 (capped)	no
5	H1 NDBI-change, summer	0.435831	×4	1.0 (capped)	no
6	H3 border-prox., lights, summer	0.265772	×3	1.0 (capped)	no
7	H1 NDBI-change, full-year	0.999999...	×2	1.0 (capped)	no
8	H1 lights-change, summer	0.999994	×1	1.0 (capped)	no

Ye table khud, step-by-step, poore Holm-Bonferroni arithmetic ko haath se dobara kar ke dikhata hai, taaki dikhe ki multipletests() ke andar exactly kya calculation ho rahi hai. Rank 1 aur 2 ke liye, seedha multiplication hi kaam kar gaya (raw × multiplier), kyunki dono values chhoti thi aur cap se kaafi door thi. Rank 3 ke liye bhi seedha multiplication ($0.050075 \times 6 = 0.300450$) — na 1.0 ko cross kiya, na monotonicity-rule ki zaroorat padi.

Lekin Rank 4 se ek interesting cheez shuru hoti hai: $0.443353 \times 5 = 2.2168$, jo 1.0 se zyada hai — isliye simply 1.0 par cap kar diya jaata hai, kyunki koi p-value 1.0 se upar nahi ho sakti. Rank 5 ($0.435831 \times 4 = 1.7433$) ka apna raw-multiplied value 1.0 se zyada hai, isliye wo bhi 1.0 par cap ho jaata hai. Rank 6 ($0.265772 \times 3 = 0.7973$) ka apna raw-multiplied value khud 1.0 se kam hai — lekin Holm ka monotonicity-rule kehta hai ki koi bhi rank apne se upar wale rank (chhote raw-p wale) se kam adjusted-value nahi dikha sakta. Rank 5 already 1.0 par hai, isliye Rank 6 ka final adjusted value bhi $\max(0.7973, 1.0) = 1.0$ ban jaata hai — chahe uska apna multiplication 1.0 se kam ho. Rank 7 aur 8 mein bhi yehi cheez repeat hoti hai — Rank 7 ka apna $0.999999... \times 2 \approx 2.0$ (cap se upar), aur Rank 8 ka apna $0.999994 \times 1 \approx 0.999994$ (cap se thoda neeche), lekin monotonicity ki wajah se ye bhi 1.0 par hi settle hota hai, kyunki upar wale sabhi ranks already 1.0 par pahunch chuke hain.

EXAMPLE — Ye Exactly Wahi Cheez Hai Jo Entry 16 Ne Pakdi

Ab dekho ki Entry 16 ki bug is table mein kahan chhupi thi: agar koi casually likh de ki 'sabhi 6 non-significant results ke adjusted p exactly 1.000 hain,' toh Rank 3 (H1 lights-change, full-year, adjusted = 0.300450) chhoot jaata — wo bhi non-significant hai ($0.300450 > 0.05$), lekin uska adjusted value 1.0 nahi hai, balki genuinely 0.300 hai, kyunki wo procedure mein bahut jaldi (rank 3 par) aata hai, jab multiplier abhi bhi kaafi bada (6) hai aur raw p-value khud bahut kam (0.050075) hai — bas thoda hi $1.0/6$

= 0.1667 se zyada. Ye number is baat ka honest signal hai ki H1 ka full-year lights-change test doosre 5 non-significant tests jitna 'clearly non-significant' nahi hai — ye ek borderline-non-significant hai, sirf Holm correction ke baad hi na-significant ban gaya, jabki uska raw p (0.050075) already 0.05 se bahut hi thoda upar tha.

Do tests significant reh gaye Holm correction ke baad bhi — H3 border-proximity NDBI (summer) aur H3 border-proximity lights (full-year), dono apne rank 1 aur 2 ki wajah se, jahan raw p-values already itne chhote the (scientific notation mein, 10^{-6} aur 10^{-5} ki range mein) ki multiply hone ke baad bhi 0.05 se kaafi neeche reh gaye. Ye is study ke do sabse robust, sabse-kam-doubtful headline findings hain.

Part B — spatial_moran_and_h3_correction.py

Ab jo do tests Holm correction ke baad bhi significant reh gaye — H3 ka NDBI-proximity (summer) aur H3 ka lights-proximity (full-year) — unke liye ek doosra, bilkul alag tarah ka sawaal poochha jaata hai: kya un 251 core-sample villages ko sach mein 251 independent observations ki tarah treat karna sahi hai? Ya paas-paas ke villages ek-doesre se milte-julte outcomes dikhate hain, sirf apni geography ki wajah se — jisse asli, effective sample size 251 se kam ho jaaye, aur p-value jitna hona chahiye usse zyada optimistic ho jaaye?

Docstring Aur Imports

```
"""
```

```
Section 6.11 follow-through: reproduces the published Moran's I values as a
sanity check, then computes the spatially-corrected significance for H3
that was flagged as not done yet. Approach: calibrate a SAR(1) field to
match the observed Moran's I (k=8 NN weights), simulate a bunch of those,
and see how often a purely-spatial field correlates with border distance
as strongly as the real data does. Simplified version of Clifford et al./
Dutilleul's correction, not the closed-form formula itself.
```

```
"""
```

```
import json
```

```
import numpy as np
```

```
import pandas as pd
```

```
from scipy import stats
```

Purpose: Docstring apni poori strategy ek jagah bata deta hai: pehle, paper mein already published Moran's I values ko dobara reproduce karke integrity-check karna (kya humara code wahi number deta hai jo paper mein likha hai). Fir, ek naya calculation karna — H3 ke un tests ke liye jo abhi 'flagged as not done yet' the, unke spatially-corrected significance nikaalna. Approach clearly explain kiya gaya hai: ek

SAR(1) — Spatial AutoRegressive, order 1 — model ko calibrate karna taaki wo observed Moran's I ko match kare, phir bahut saare aise purely-spatial (bina kisi real causal-effect ke) fields simulate karna, aur dekhna ki unme se kitne 'by chance' hi asli data jitna strong correlation dikha dete hain border distance ke saath. Aakhri sentence honest hai — ye Clifford et al./Dutilleul's actual closed-form mathematical formula nahi hai, balki uska ek simplified, simulation-based version. Syntax: import json — result ko JSON file mein save karne ke liye. import numpy as np — matrix operations ke liye (PDF 14 mein dekha tha @ operator wagera). import pandas as pd — dataframes. from scipy import stats — Spearman correlation ke liye, isi PDF ke Part A mein bhi dekha.

Constants

```
DISTANCES = "data/processed/border_optics_master_villages_with_distance.csv"
RESULT_FILES = {
    "full_year": "data/processed/border_optics_village_results_analyzed.csv",
    "summer": "data/processed/border_optics_village_results_summer_analyzed.csv",
}
K = 8
N_PERM_MORAN = 999
N_SIM_H3 = 2000
RNG_SEED = 42
OUT_JSON = "outputs/spatial_moran_h3_correction_results.json"
```

Purpose: Sabhi zaroori settings ek jagah — file-paths, K=8 (kitne nearest-neighbors, yaani 'paas ke kitne villages,' spatial-weight banana ke liye use hote hain), N_PERM_MORAN=999 (Moran's I ka apna significance test kitni random shuffles se banaya jaayega), N_SIM_H3=2000 (spatial correction ke liye kitne simulated fields banaye jaayenge), aur ek fixed RNG_SEED=42 (taaki har baar chalane par exactly wahi 'random' numbers milein, reproducibility ke liye). Syntax: Sabhi CAPITAL_LETTER naam Python convention hain 'constant' batane ke liye — ye values script chalte waqt kabhi badalti nahi hain, sirf top par ek jagah define ki jaati hain taaki poore file mein consistently use ho sakein.

haversine_km() — Do Coordinates Ke Beech Ki Sachi, Curved-Earth Distance

```
def haversine_km(lat1, lon1, lat2, lon2):
    R = 6371.0088
    lat1, lon1, lat2, lon2 = map(np.radians, (lat1, lon1, lat2, lon2))
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    return 2 * R * np.arcsin(np.sqrt(a))
```

Purpose: Ye function do (latitude, longitude) points ke beech ki asli distance kilometers mein nikaalta hai, Earth ki gol shape ko account karte hue — seedhi-line

(Euclidean) distance nahi, balki 'haversine formula' jo Earth ko ek sphere maan kar great-circle distance nikaalta hai. Ye bilkul wahi concern hai jo PDF 16 ke Part A mein Entry 18 ke UTM-vs-geodesic fix mein dekha — yahan bhi wahi lesson repeat ho rahi hai: geographic distances ke liye Earth ki curvature ko ignore karna galat numbers de sakta hai, khaas kar bade area (jaise poora North-East India border-belt) ke liye. Yahan ye function coordinate-based nearest-neighbor weights banana ke liye use hoga, seedhe distance-to-border ke liye nahi. Syntax: $R = 6371.0088$ — Earth ka average radius, kilometers mein, ek well-known constant. `map(np.radians, (lat1, lon1, lat2, lon2))` — ek function ko multiple values par ek saath apply karna (degrees ko radians mein badalna, jo trigonometric functions [sin, cos] ke liye zaroori hai), aur multiple-assignment se sabko wapas unke apne naam mein store karna. `np.sin()`, `np.cos()`, `np.arcsin()`, `np.sqrt()` — trigonometric aur algebraic functions, numpy arrays par element-wise kaam karte hain, matlab `lat1` ek poora array ho sakta hai (sab villages ke liye ek saath), function phir bhi kaam karega, har element par alag-alag.

load_core() — Distance Aur Results Ko Jodna, Sirf Core Sample Rakhna

```
def load_core(window):
    distances = pd.read_csv(DISTANCES)
    results = pd.read_csv(RESULT_FILES[window]).drop(columns=["latitude", "longitude"],
    errors="ignore")
    merged = results.merge(
        distances[["village_id", "latitude", "longitude", "distance_to_border_km"]],
        on="village_id", how="inner",
    )
    return merged[merged["is_core_sample"] == True].reset_index(drop=True)
```

Purpose: Ek window (full_year ya summer) ke liye results file padhta hai, uske apne latitude/longitude columns (agar hon) hata deta hai (kyunki hum distances file ke coordinates use karenge, taaki source consistent rahe), phir village_id ke through distances file se latitude/longitude/distance_to_border_km jod deta hai. Aakhri mein, sirf core-sample rows rakhta hai, aur index ko fresh, 0-se-shuru numbering deta hai. **Syntax:** `.drop(columns=[...], errors="ignore")` — specified columns hata dena, aur agar wo columns already exist hi nahi karte, toh error na de (normal `drop()` error deta agar column missing ho). `.reset_index(drop=True)` — dataframe ke row-index ko fresh 0,1,2,... se numbering, purana index column ke roop mein save kiye bina (`drop=True`) — filtering ke baad ye zaroori hota hai kyunki filter karne se index mein gaps aa jaate hain.

build_knn_weights() — Har Village Ke '8 Sabse Paas Ke Padosi' Dhoondhna

```
def build_knn_weights(lat, lon, k=K):
    n = len(lat)
```

```

W = np.zeros((n, n))
for i in range(n):
    d = haversine_km(lat[i], lon[i], lat, lon)
    d[i] = np.inf
    nn = np.argsort(d)[:k]
    W[i, nn] = 1.0
row_sums = W.sum(axis=1, keepdims=True)
row_sums[row_sums == 0] = 1.0
return W / row_sums

```

Purpose: Ye function ek $n \times n$ 'spatial weight matrix' W banata hai, jahan n = kitne villages hain (yahan 251). Har row i ek village ko represent karti hai, aur us row mein sirf uske $k=8$ sabse paas ke villages ka column 1.0 hota hai, baaki sab 0. Har village ke liye, `haversine_km()` se sabhi doosre villages ki distance nikaali jaati hai, khud-se-khud ki distance ko infinity bana diya jaata hai (taaki koi village apna khud ka neighbor na bane), fir sabse chhoti 8 distances wale villages ko dhoondh liya jaata hai. Aakhri mein, har row ko normalize kar diya jaata hai (row-sum se divide karke) — taaki har row ka total exactly 1.0 ho, aur har weight $1/8$ ban jaaye. Ye 'row-normalized k-nearest-neighbor weight matrix' hai, spatial statistics mein ek bahut standard tareeka 'kaun kisse paas hai' ko numerically represent karne ka. Syntax: `np.zeros((n, n))` — ek $n \times n$ matrix banana, sab values 0 se shuru. `for i in range(n):` — har row (village) ke liye loop. `d[i] = np.inf` — matrix/array ke ek specific index ki value ko infinity set karna, taaki wo 'sabse door' maana jaaye aur kabhi top-k mein na aaye. `np.argsort(d)` — ek array ko sort karne ke bajaye, sorted order mein unke ORIGINAL indices return karta hai (jaise agar $d=[5,1,3]$, toh `argsort` de $[1,2,0]$, kyunki $d[1]=1$ sabse chhota hai). `[:k]` — pehle k items lena, slicing syntax. `W[i, nn] = 1.0` — fancy indexing (PDF 14 mein bhi dekha tha), row i ke un sabhi columns (jo `nn` list mein hain) ko ek saath 1.0 set karna. `W.sum(axis=1, keepdims=True)` — har row ka total nikaalna, `keepdims=True` se result 2D shape mein rehta hai $(n,1)$ taaki aage division seedhe broadcasting se ho sake. `row_sums[row_sums == 0] = 1.0` — divide-by-zero se bachne ke liye, agar koi row-sum 0 ho (jo theoretically nahi hona chahiye yahan, lekin defensive coding hai) toh use 1 bana do.

EXAMPLE — Ek Class Mein Dosti Ki Analogy — Weight Matrix Ka Matlab

Socho ek class mein 251 students hain, aur har student ke apne 8 sabse kareebi dost hain (jo unke ghar ke sabse paas rehte hain). Agar hum poochna chahein 'kya paas rehne wale students milte-julte marks laate hain,' toh hume ek tareeka chahiye har student ko unke 8 dosto se jodne ka, taaki calculation mein sirf unhi ka asar count ho, poori class ka nahi. Weight matrix W exactly ye kaam karta hai — har row ek student, usme sirf uske 8 dosto ka column non-zero ($1/8$ each, normalize karne ke baad), baaki sab zero. Ye matrix aage Moran's I calculate karne mein use hoga, jo 'paas ke logon ke outcomes kitne milte-julte hain' ko ek single number mein summarize karta hai.

morans_i() — Spatial Clustering Ko Ek Single Number Mein Summarize Karna

```
def morans_i(x, W):  
    z = x - x.mean()  
    num = z @ W @ z  
    den = (z ** 2).sum()  
    return num / den if den != 0 else np.nan
```

Purpose: Moran's I statistic ka poora formula sirf 4 lines mein. Pehle, x (jaise ndbi_change ki values, sabhi 251 villages ke liye) ko 'mean-centered' kiya jaata hai — $z = x - x.mean()$, matlab har value se poore array ka average ghata diya jaata hai, taaki z mein positive values 'average se upar' aur negative 'average se neeche' represent karein. Numerator ($z @ W @ z$) ye check karta hai ki jab ek village apne average se upar hai, kya uske spatial-neighbors (jo W ne define kiye) bhi average se upar hain — agar haan, toh ye product positive hoga aur Moran's I positive/high banega, jo 'clustering' (paas-paas wale similar hain) dikhata hai. Denominator $(z^2).sum()$ sirf ek normalizing factor hai, taaki final value ek consistent scale par rahe (roughly -1 se +1 ke beech, jaise correlation coefficient). Syntax: $x.mean()$ — array ka average, numpy/pandas ka built-in method. $z @ W @ z$ — matrix multiplication ka chain, PDF 14 mein dekha gaya @ operator; yahan z ek n-length vector hai, W ek $n \times n$ matrix, toh $z @ W$ ek naya n-length vector deta hai (har village ke apne neighbors ka weighted-average z-value), aur usko phir se z se multiply (dot product) karne se ek single number milta hai — 'apne z-value aur apne neighbors ke average z-value ke beech ka overall alignment.' $(z^2).sum()$ — har element ko square karke sabka total, variance jaisi cheez. Ternary conditional expression — value if condition else other_value, ek line mein divide-by-zero se bachna.

EXAMPLE — Positive, Negative, Aur Zero Moran's I Ka Matlab

Agar Moran's I positive hai (jaise 0.346, summer NDBI ke liye), toh iska matlab hai spatial clustering hai — paas-paas ke villages ek-doesre se milte-julte outcomes dikhate hain (jo dono average se upar hain ya dono neeche). Agar Moran's I zero ke kareeb hota, toh iska matlab hota koi spatial pattern nahi — outcomes randomly, bina geography ke asar ke, phaila hue hain. Agar Moran's I negative hota (jo yahan nahi mila), toh iska matlab hota 'checkerboard' pattern — paas ke villages ek-doesre se systematically opposite (ek upar, doosra neeche) hote. Yahan dono outcomes (summer NDBI-change: $I=0.346$; full-year lights-change: $I=0.076$) positive hain, jismein summer NDBI ka clustering kaafi strong hai (0.346, jo -1-se-+1 scale par ek badi value hai), aur lights ka comparatively chhota lekin phir bhi real (0.076).

morans_i_permutation_p() — Ye Clustering Random Chance Se Nahi Hai, Ye Kaise Pata Chala

```
def morans_i_permutation_p(x, W, n_perm=N_PERM_MORAN, seed=RNG_SEED):  
    rng = np.random.RandomState(seed)
```

```

obs = morans_i(x, W)
n = len(x)
perm_vals = np.empty(n_perm)
for i in range(n_perm):
    perm_vals[i] = morans_i(rng.permutation(x), W)
p = (np.sum(np.abs(perm_vals) >= np.abs(obs)) + 1) / (n_perm + 1)
return obs, p, perm_vals.mean(), perm_vals.std()

```

Purpose: Ye function pehle asli, observed Moran's I nikaalta hai (obs), phir 999 baar x ki values ko randomly shuffle karta hai (villages ke beech unke outcomes ko randomly redistribute karke, unki geography ko waisa hi rakhte hue) aur har baar Moran's I dobara compute karta hai. Ye 999 'agar geography ka koi asar na hota' wale scenarios ka ek null distribution banata hai. Fir p-value nikaalta hai: in 999 random-shuffle wale Moran's I mein se kitne, apni absolute value mein, asli observed Moran's I jitne ya usse zyada extreme hain. Ye exactly wahi 'permutation test' logic hai jo PDF 13 mein randomization_inference() mein pehli baar dekha tha — bas yahan permutations 999 hain (2000 ke bajaye), aur target statistic Moran's I hai, Spearman correlation nahi.

Syntax: np.random.RandomState(seed) — PDF 13/14 mein bhi dekha, ek reproducible random-number-generator object. np.empty(n_perm) — ek array pehle se allocate karna (memory efficiency ke liye), values abhi undefined/garbage hain, loop mein fill hongi. rng.permutation(x) — x ki values ko randomly shuffle karna (rows ka order badalna, values wahi rehte hain, bas kisko kaunsi value milegi wo randomly badal jaata hai). np.abs() — absolute value, negative ko positive bana dena, taaki dono taraf (bahut positive ya bahut negative) extreme values count ho sakein. '+1' pseudo-count formula ((count + 1)/(n_perm + 1)) — PDF 13 mein bhi dekha gaya convention, taaki p-value kabhi bhi exactly zero na ho (jo theoretically galat hota, kyunki asli data khud bhi ek 'possible' permutation hai). perm_vals.mean(), perm_vals.std() — is null-distribution ka apna average aur spread, reporting ke liye.

calibrate_sar_phi() — Ek 'Sirf-Geography' Wala Fake Data Banane Ka Tareeka Dhoondhna

```

def calibrate_sar_phi(target_I, W, n, seed=RNG_SEED, draws_per_eval=60):
    # bisect on phi so simulated SAR(1) field's Moran's I matches target_I
    rng = np.random.RandomState(seed)
    I_n = np.eye(n)

    def mean_I_at(phi):
        M = np.linalg.inv(I_n - phi * W)
        vals = np.empty(draws_per_eval)
        for j in range(draws_per_eval):
            eps = rng.standard_normal(n)
            y = M @ eps
            vals[j] = morans_i(y, W)
        return vals.mean()

```


Purpose: Ye function ka pehla hissa ek helper (`mean_I_at`) define karta hai jo diya gaya ϕ (kitna 'spatial dependence,' 0 se 1 ke beech ek number) lekar, ek SAR(1) — Spatial AutoRegressive — formula se random fields banata hai aur unka average Moran's I return karta hai. SAR(1) formula khud ye kehta hai: $y = (I - \phi * W)^{-1} @ \text{eps}$, jahan eps bilkul random, spatial-structure-free noise hai, aur $(I - \phi * W)^{-1}$ us noise ko 'smear' karta hai apne neighbors ke through, jitna ϕ zyada hota hai, utna zyada smearing — matlab utna zyada spatial clustering. Comment khud poora goal bata deta hai: 'bisect on ϕ so simulated SAR(1) field's Moran's I matches target_I' — hum wo ϕ dhoondhna chahte hain jo, average mein, hamare asli data jitna hi Moran's I produce kare. Syntax: `np.eye(n)` — identity matrix, $n \times n$, diagonal par 1 baaki sab 0 (matrix-algebra ka 'ek se multiply karo, kuch nahi badlega' wala element). `np.linalg.inv(matrix)` — matrix ka inverse nikaalna (PDF 14 mein bhi dekha, wahan $X.T @ X$ ka inverse tha, yahan $(I - \phi * W)$ ka). `rng.standard_normal(n)` — n random numbers, standard normal distribution se (mean 0, spread 1) — ye 'eps,' bilkul random, bina kisi spatial pattern ke, shuruati noise hai. `M @ eps` — matrix-vector multiplication, is random noise ko spatial-structure de deta hai. Nested function definition — ek function (`mean_I_at`) doosre function (`calibrate_sar_phi`) ke andar define kiya gaya hai, jo Python mein allowed hai; ye 'closure' outer function ke variables (`rng`, `I_n`, `W`) ko directly access kar sakta hai, bina unhe explicitly pass kiye.

```
lo, hi = 0.0, 0.97
lo_val, hi_val = mean_I_at(lo), mean_I_at(hi)
if target_I <= lo_val:
    return lo
if target_I >= hi_val:
    return hi
for _ in range(18):
    mid = (lo + hi) / 2
    mid_val = mean_I_at(mid)
    if mid_val < target_I:
        lo = mid
    else:
        hi = mid
return (lo + hi) / 2
```

Purpose: Ye 'bisection search' hai — ek classic tareeka kisi function ka input dhoondhne ka jab tumhe pata ho ki output monotonically (seedha, bina ups-and-downs ke) badhta hai jaise input badhta hai. $\phi=0$ par kitna Moran's I milta hai (`lo_val`, jo lagbhag 0 hona chahiye, kyunki $\phi=0$ ka matlab hai koi spatial dependence hi nahi) aur $\phi=0.97$ par kitna (`hi_val`, jo bahut high hona chahiye) — dono check kiye jaate hain edge-cases ke liye. Fir 18 baar, range ke beech (`mid`) point try kiya jaata hai: agar wahan ka Moran's I target se kam hai, toh answer upper-half mein hai (`lo` ko `mid` bana do); agar zyada hai, toh lower-half mein hai (`hi` ko `mid` bana do). Har iteration range ko aadha kar deta hai, isliye 18 iterations ke baad range itni

chhoti ho jaati hai ki mid-point practically exact answer ban jaata hai ($2^{18} = 262,144$ — original 0.97 wide range ko itni baar aadha karne se precision bahut high ho jaati hai). Syntax: Tuple unpacking — `lo, hi = 0.0, 0.97` ek line mein do variables assign karna. `for _ in range(18):` — underscore (`_`) ka use jab loop variable khud kabhi use nahi hoga, sirf 'ye 18 baar chalao' matter karta hai. `if/elif` jaisa branching — `mid_val` ko target se compare karke `lo` ya `hi` ko update karna, classic binary-search pattern.

EXAMPLE — Bisection Search Ki Ek Choti Analogy — 'Higher Ya Lower' Wala Number Guessing Game

Ye bilkul wahi tareeka hai jo 'main ek number soch raha hoon 1 se 100 ke beech, guess karo' wale game mein sabse smart strategy hai — har baar seedha beech ka number guess karo (jaise 50), aur 'higher' ya 'lower' ka jawaab sunke apni range aadhi kar do (agar 'higher' bola, toh ab range 51-100). Bas 7 guesses mein 100 numbers ke beech se sahi answer mil jaata hai, kyunki har guess range ko aadha kar deta hai. `calibrate_sar_phi()` bilkul yehi khel khel raha hai, sirf 'number' ki jagah `phi` (0 se 0.97 ke beech) hai, aur 'higher/lower' ka jawaab `mean_I_at(mid)` function deta hai — 'is `phi` par jo Moran's I milta hai, wo target se zyada hai ya kam.'

`spatial_null_h3()` — Ye Poori File Ka Core, Sabse Zaroori Function

```
def spatial_null_h3(window, metric):
    df = load_core(window).dropna(subset=[metric, "distance_to_border_km", "latitude",
    "longitude"])
    n = len(df)
    lat = df["latitude"].values
    lon = df["longitude"].values
    x = df[metric].values
    d = df["distance_to_border_km"].values

    W = build_knn_weights(lat, lon)
    obs_I, moran_p, perm_mean, perm_sd = morans_i_permutation_p(x, W)
    obs_rho, obs_p_naive = stats.spearmanr(d, x)
```

Purpose: Function ka pehla hissa data taiyaar karta hai — `window` aur `metric` (jaise 'summer', 'ndbi_change') ke liye core-sample data load karke, sabhi zaroori columns ki missing-values wali rows hata deta hai, phir har zaroori column ko ek plain numpy array mein extract karta hai. Fir spatial weight matrix banata hai, Moran's I aur uska permutation-based p-value nikaalta hai (upar ke functions se), aur is `metric` ka apna asli, naive (uncorrected) Spearman correlation-aur-p-value bhi nikaal leta hai border-distance ke saath — yehi wahi `naive_p` hai jo Holm correction (Part A) mein bhi use hua tha. Syntax: `.dropna(subset=[...])` — PDF 15/16 mein bahut baar dekha, missing-value rows hatana. `.values` — pandas Series/column ko plain numpy array mein badalna, jo matrix-algebra operations ke liye zaroori hai. Multiple return-values unpack karna, ek saath — `obs_I, moran_p, perm_mean, perm_sd = morans_i_permutation_p(x, W)`.

```

phi = calibrate_sar_phi(obs_I, W, n)

rng = np.random.RandomState(RNG_SEED + 1)
I_n = np.eye(n)
M = np.linalg.inv(I_n - phi * W)
sim_rhos = np.empty(N_SIM_H3)
sim_Is = np.empty(N_SIM_H3)
for i in range(N_SIM_H3):
    eps = rng.standard_normal(n)
    y_sim = M @ eps
    sim_rhos[i] = stats.spearmanr(d, y_sim)[0]
    sim_Is[i] = morans_i(y_sim, W)

p_spatial = (np.sum(np.abs(sim_rhos) >= np.abs(obs_rho)) + 1) / (N_SIM_H3 + 1)

```

Purpose: Yahan asli 'spatial correction' hoti hai. Pehle, `calibrate_sar_phi()` se wo ϕ dhoondha jaata hai jo asli data ke observed Moran's I (`obs_I`) ko match karta hai. Fir 2000 baar: ek bilkul random, spatial-structure-free noise (`eps`) banao, use SAR(1) formula se 'spatial texture' do (`y_sim = M @ eps`) — matlab ek fake dataset banao jismein koi asli causal-relationship border-distance ke saath nahi hai, lekin geography ki wajah se wahi tarah ka clustering hai jitna asli data mein hai. Har aise fake dataset ke liye, dekho ki border-distance ke saath uska Spearman correlation kitna aata hai (`sim_rhos[i]`) — aur zaroorat ho toh verify karo ki uska Moran's I bhi target jaisa hi hai (`sim_Is[i]`, jo print step mein calibration ki quality check karega). Aakhri mein, `p_spatial` nikaalta hai: in 2000 purely-spatial (bina kisi real border-effect ke) fields mein se kitne, apni absolute value mein, asli, observed Spearman rho jitna ya usse zyada extreme correlation dikhaate hain, border-distance ke saath. Ye hi asli sawaal ka jawaab hai: 'agar sirf geography (bina kisi treatment-effect ke) itna hi clustering produce karti, toh kitni baar wo akele hi itna strong border-distance correlation dikha deti?' Syntax: `np.random.RandomState(RNG_SEED + 1)` — ek nayi seed, thodi alag Moran's I permutation test ki seed se (`RNG_SEED` vs `RNG_SEED+1`), taaki dono randomizations independent rahen, ek-doesre se leak na karen. `stats.spearmanr(d, y_sim)[0]` — `spearmanr()` do values return karta hai (correlation, p-value); `[0]` indexing se sirf correlation-value nikaali ja rahi hai (index `[1]` hota p-value, jo yahan nahi chahiye kyunki uska apna p-value meaningless hai — hum sirf real `p_spatial` simulation se khud nikaalenge). Same '+1' pseudocount formula jo Moran's permutation test mein bhi dikha.

```

return {
    "window": window, "metric": metric, "n": n,
    "moran_i_observed": float(obs_I), "moran_i_permutation_p": float(moran_p),
    "moran_i_permutation_null_mean": float(perm_mean),
    "moran_i_permutation_null_sd": float(perm_sd),
    "spearman_rho": float(obs_rho), "naive_p": float(obs_p_naive),
    "calibrated_sar_phi": float(phi), "calibrated_sim_mean_moran_i":
float(sim_Is.mean()),

```

```

    "spatially_corrected_p": float(p_spatial),
    "n_sim": N_SIM_H3,
}

```

Purpose: Function ka aakhri hissa sabhi computed values ko ek single, well-named dictionary mein pack karta hai — asli Moran's I aur uski significance, asli Spearman rho aur uska naive p-value, calibrated phi, calibration quality-check (calibrated_sim_mean_moran_i — kitna close ye target ke paas gaya), aur sabse zaroori, spatially_corrected_p — final answer. **Syntax:** float(numpy_value) — numpy ke apne number-types (jaise np.float64) ko plain Python float mein badalna, taaki JSON serialization mein koi problem na aaye. Dictionary literal with many keys — is poore series mein bahut baar dekha gaya, ek function ke poore result ko ek named, readable structure mein return karna, tuple ki tarah unclear positional values ke bajaye.

main() — Do Steps: Integrity-Check, Fir Asli Correction

```

def main():
    print("=== Step 1: reproduce this study's own published Moran's I values (integrity
    check) ===\n")
    checks = [
        ("summer", "ndbi_change", "\u00a76.11 claims I=0.346, p=0.001"),
        ("full_year", "lights_change", "\u00a76.11 claims I=0.076, p=0.004"),
    ]
    reproduction = {}
    for window, metric, claim in checks:
        df = load_core(window).dropna(subset=[metric, "latitude", "longitude"])
        W = build_knn_weights(df["latitude"].values, df["longitude"].values)
        obs_I, p, pm, psd = morans_i_permutation_p(df[metric].values, W)
        print(f"{window}/{metric} (published: {claim})")
        print(f"Reproduced: I={obs_I:.3f}, permutation p={p:.4f} (n={len(df)}, k=8 NN
        weights)\n")
        reproduction[f"{window}_{metric}"] = {"reproduced_I": obs_I, "reproduced_p": p,
        "published_claim": claim}

```

Purpose: Step 1 ek honesty-check hai, asli correction se pehle: paper ke §6.11 mein jo Moran's I values already published hain (summer NDBI: I=0.346, p=0.001; full-year lights: I=0.076, p=0.004), unhe is fresh code-run se dobara reproduce karke dikhaya jaata hai ki wo numbers galat nahi the aur ye naya code sahi tareeke se kaam kar raha hai — is se aage jo bhi naya spatially_corrected_p nikalega, uspar zyada bharosa kiya ja sakta hai, kyunki underlying Moran's I calculation khud verified hai. **Syntax:** for window, metric, claim in checks: — 3-tuples ki list par loop, teeno values ek saath unpack. f-string ke andar \u00a7 jaisa Unicode escape — § symbol (section sign) ko safely represent karna bina directly non-ASCII character file mein likhe, jo reportlab jaise render-ers mein compatibility-safe rehta hai.

```

combos = [
    ("summer", "ndbi_change", "H3 NDBI-proximity, summer (the new, flagged finding)"),
    ("full_year", "lights_change", "H3 lights-proximity, full-year (pre-existing
Holm-significant result)"),
]
corrections = {}
for window, metric, desc in combos:
    r = spatial_null_h3(window, metric)
    print(f"--- {desc} ---")
    print(f" n={r['n']} observed rho={r['spearman_rho']:+.5f} naive p={r['naive_p']:.3g}")
    print(f" Moran's I of outcome = {r['moran_i_observed']:.4f} (permutation
p={r['moran_i_permutation_p']:.4f})")
    print(f" Calibrated SAR phi = {r['calibrated_sar_phi']:.4f} "
          f"(simulated fields' mean I = {r['calibrated_sim_mean_moran_i']:.4f}, target
{r['moran_i_observed']:.4f})")
    print(f" Spatially-corrected p (2000 spatially-autocorrelated null draws) =
{r['spatially_corrected_p']:.5f}")
    print(f" {'STILL significant' if r['spatially_corrected_p'] < 0.05 else 'NO LONGER
significant'} ")
    print(f"at alpha=0.05 after spatial correction\n")
    corrections[f"{window}_{metric}"] = r

with open(OUT_JSON, "w") as f:
    json.dump({"moran_reproduction": reproduction, "h3_spatial_correction":
corrections}, f, indent=2)
print(f"Saved {OUT_JSON}")

```

Purpose: Step 2 asli kaam karta hai — bilkul wahi do combinations jo Part A ke Holm correction mein significant reh gaye the (summer NDBI-proximity, full-year lights-proximity), unke liye `spatial_null_h3()` call karke poora corrected-result dictionary banaya jaata hai, aur console par har zaroori number (observed rho, naive p, Moran's I, calibrated phi, final spatially-corrected p, aur ek seedha 'STILL/NO LONGER significant' verdict) print hota hai. Aakhri mein, dono steps ke results (reproduction aur corrections) ek hi JSON file mein save ho jaate hain, nested dictionary ki tarah. Syntax: `for window, metric, desc in combos:` — 3-tuples ki list par loop, teeno values ek saath unpack. Dictionary ke values access karna f-string ke andar — `f"{r['n']}"` — dictionary lookup seedha string-formatting ke andar. Ternary/conditional expression ek f-string ke andar — `'STILL significant' if condition else 'NO LONGER significant'` — PDF 15 mein bhi is tarah ka pattern dekha tha, bare-verdict print karne ke liye. `with open(path, "w") as f:` — file safely open aur close karna (Python ka 'context manager,' block khatam hote hi file automatically close ho jaati hai, chahe koi error bhi aaye). `json.dump(dict, f, indent=2)` — Python dictionary ko JSON format mein file mein likhna, `indent=2` se human-readable formatting milta hai.

History — `spatial_moran_and_h3_correction.py` Ke Peeche Ki Kahani

EXAMPLE — Woh Din Jab Ye Pehli Baar 'Identified, Not Yet Tested' Se 'Tested' Bana — Entry 27

Bahut pehle se, paper ke §6.10 (Threats-to-Validity table) mein ek row thi jo spatial autocorrelation ko sirf 'identified, not yet tested' kehti thi — matlab researcher ko pata tha ki ye ek theoretical concern hai, lekin actually test nahi kiya gaya tha. Development Log Entry 27 mein ye pehli baar sach mein test kiya gaya — libpysal/esda jaise specialized spatial-stats packages is sandbox mein install nahi the, isliye Moran's I ko haath se implement kiya gaya (bilkul wahi 'jab package available na ho, khud se likho' pattern jo PDF 14 mein OLS ke liye bhi dekha tha). Result clear tha: summer NDBI-change ka Moran's I = 0.346 (null mean -0.002, SD 0.029 ke against), permutation p = 0.001; full-year lights-change ka I = 0.076 (null mean -0.004, SD 0.020), p = 0.004. Dono real, positive, significant spatial clustering — paas-paas ke villages ke outcomes chance se zyada milte-julte hain.

EXAMPLE — Ek Zaroori Extra Check — Ye Sirf Distance-Effect Ka Doosra Naam Nahi Hai

Entry 27 mein ek aur zaroori check ki gayi thi jo is script ki apni morans_i() function se hi related hai, chahe wo alag se run hui thi: sirf ye dekhna kaafi nahi tha ki outcomes mein spatial clustering hai — ye bhi confirm karna zaroori tha ki ye clustering sirf border-distance ke asar ka doosra naam nahi hai (kyunki border ke paas ke villages geography mein bhi ek-doesre ke paas hote hain, isliye distance-effect aur spatial-clustering overlap kar sakte hain). Entry 27 ne summer NDBI-change ke RESIDUALS (yaani, distance-to-border ka linear-trend hataane ke baad jo bacha) par bhi Moran's I recompute kiya — aur residuals mein bhi I = 0.272, p = 0.001 mila. Iska matlab: paas-paas ke villages sirf apni shared border-distance ki wajah se milte-julte nahi hain — unme aur bhi kuch shared hai (shayad local economy, infra-structure, ya micro-climate) jo unhe similar banata hai, border-distance se independent tareeke se. Ye is baat ki extra pushti hai ki spatial-correction genuinely zaroori thi, na ki border-distance ke asar ko double-count karna.

EXAMPLE — Entry 32 — Pehli Baar Asli Spatial-Corrected p-value Nikala Gaya

Development Log Entry 32 mein hi pehli baar §6.11 ka doosra hissa — 'spatially-corrected significance' — actually compute kiya gaya, isi spatial_moran_and_h3_correction.py se. Entry khud likhta hai ki ye method 'the spatial permutation null that respects the observed clustering structure' hai, jo §6.11 ne khud Dutilleul (1993)'s closed-form formula ke ek alternative ki tarah naam liya tha — is entry mein wahi banaya gaya, closed-form formula khud nahi. Results: summer NDBI-proximity ka naive p (0.0000026) spatial-correction ke baad p=0.02949 ban jaata hai — abhi bhi 0.05 se neeche, lekin sirf thoda-sa, jahan raw number bilkul unambiguous dikh raha tha. Full-year lights-proximity ka naive p (0.0000548) correction ke baad p=0.00350 ban jaata hai — comfortably significant, dono tareeke se. Entry khud is farak ko explicitly zaroori maanta hai: 'a genuinely important distinction for how much weight the paper's two surviving H3 findings deserve

relative to each other' — matlab dono results 'significant' hi rehte hain, lekin unka confidence-level ek jaisa nahi hai, aur ye baat honestly report ki gayi, ek single 'both survive' statement mein chhupaayi nahi gayi.

EXAMPLE — Entry 47 — Holm Aur Spatial Correction Sequential Nahi, Parallel Hain

Development Log ki sabse recent, sabse subtle correction is PDF ke liye Entry 47 mein hai. Ek doosri, independent external review ne paper ke §4.9 Threats-to-Validity table mein ek genuine description-mismatch pakda: table ye claim kar rahi thi ki Holm aur spatial corrections, H3 ke liye, SEQUENTIAL hain — 'Holm adjusted→spatially corrected... computed on top of the Holm result, not instead of it.' Lekin jab `spatial_moran_and_h3_correction.py` ka `spatial_null_h3()` function seedha padha gaya (jo hum abhi upar poora dekh chuke hain), usme `obs_rho` — asli, NAIVE Spearman correlation — use hota hai, kisi Holm-adjusted p-value ka koi reference nahi hai poore function mein. Matlab, Holm correction (Part A) aur spatial correction (Part B), dono hi ek hi naive Spearman rho ke, do PARALLEL, alag-alag tarah ke, tests hain — ek doosre ke result par nahi bana hua, ek doosre ke against checked. Fix sirf table ki wording thi — 'computed on top of' hataakar sahi tareeka describe kiya gaya; practical conclusion (spatially corrected p final, sabse conservative, reported figure hai) already sahi tha aur badalne ki zaroorat nahi thi.

EXAMPLE — Parallel Ka Matlab Samajhne Ke Liye Ek Analogy

Socho ek student ka exam-paper do alag teachers check kar rahe hain, dono ek-dosre ka result dekhe bina, apne-apne standard se: ek teacher 'kitne total marks kate' ka strict rule follow karta hai (Holm — 'kitne tests chalaye gaye, us hisaab se threshold sakht karo'), doosra teacher poochta hai 'kya ye answer genuinely unique/original hai, ya kisi doosre student se milta-julta hai' (spatial correction — 'kya ye result genuinely independent observation se aaya, ya paas-paas ke villages ek-dosre ki 'copy' kar rahe hain'). Dono teachers apne-apne tareeke se, alag-alag angle se, wahi ek raw answer ko check kar rahe hain — ek doosre ke marks ko apne calculation mein use nahi kar rahe. Yehi 'parallel, not sequential' ka matlab hai — dono corrections ek hi cheez (naive Spearman rho) par, do alag lenses se, independently apply hoti hain.

Final State — `spatial_moran_and_h3_correction.py` Ka Current Verdict

`outputs/spatial_moran_h3_correction_results.json` se seedha liye gaye current numbers — Moran's I integrity-check pehle, phir spatial correction ke poore results:

Check	Reproduced I	Reproduced p	Published Claim
Summer NDBI-change	0.346	0.001	§6.11: I=0.346, p=0.001

Full-year lights-change	0.076	0.003	§6.11: I=0.076, p=0.004
-------------------------	-------	-------	-------------------------

Dono values almost exactly match karte hain paper ke published claims se — summer NDBI ka I exact hai (0.346=0.346), aur permutation-based p thoda hi vary karta hai (0.001 vs published 0.001 — exact match; lights ke liye 0.003 vs published 0.004 — bahut kareeb, chhoti si farak sirf random-seed/permutation-count ki wajah se, kyunki ye ek Monte Carlo estimate hai, exact formula nahi). Ye integrity-check confirm karta hai ki spatial calculations sahi tareeke se kaam kar rahe hain.

Metric	n	Naive p (S pearman)	Moran's I (observed)	Calibrated phi	Spatially-corrected p	Verdict
Summer NDBI-proximity	251	2.644e-06	0.346	0.6396	0.02949	STILL significant, barely
Full-year I ights-proximity	251	5.477e-05	0.076	0.2758	0.00350	STILL significant, comfortably

Ye Part B ka aakhri, sabse zaroori takeaway hai: Holm correction (Part A) ke baad bachne wale dono H3 findings, spatial correction (Part B, ek bilkul alag, parallel tareeka) ke baad bhi significant reh jaate hain — lekin ek jaisi confidence ke saath nahi. Summer NDBI-proximity ka p, 0.02949, 0.05 ke threshold se sirf thoda hi neeche hai — ye ek 'still significant, but only just' finding hai, jiski calibrated_sar_phi (0.6396) khud is baat ka signal hai ki summer NDBI mein spatial dependence kaafi strong hai. Full-year lights-proximity, jiska phi (0.2758) kam hai (matlab kam spatial dependence), apna p=0.00350 par kaafi zyada comfortably significant reh jaata hai. Dono results asli hain, dono paper mein report hone ke laayak hain — lekin agar koi poochhe 'kaunsa zyada bharosemand hai,' jawaab clearly full-year lights-proximity hai, na ki summer NDBI-proximity, chahe unka naive p bilkul opposite order mein tha (NDBI ka naive p, 2.644e-06, lights ke naive p, 5.477e-05, se bhi chhota tha).

EXAMPLE — Ek Interessting Reversal — Jo Zyada 'Significant' Dikh Raha Tha, Wahi Zyada Fragile Nikla

Yahan ek genuinely counter-intuitive lekin important lesson hai: naive p-values ke hisaab se, summer NDBI-proximity (2.644e-06) full-year lights-proximity (5.477e-05) se dus guna zyada 'significant' dikh raha tha — chhota p-value, seedhe-seedhe padhne par, 'zyada strong evidence' ki tarah lagta hai. Lekin spatial correction ke baad, order palat jaata hai: summer NDBI (0.02949) ab lights-proximity (0.00350) se kam comfortable hai. Wajah samajhna zaroori hai — summer NDBI mein spatial clustering (phi=0.6396) khud bahut strong thi, isliye uska chhota naive p-value kaafi

hadd tak isi clustering ki wajah se 'inflate' hua tha, asli, independent evidence se zyada. Lights-proximity mein clustering kam thi ($\phi=0.2758$), isliye uska thoda-bada naive p-value 'genuine' zyada tha, kam inflated. Ye exactly wahi lesson hai jo is poore PDF ki shuruaat mein diya gaya tha: p-value ka chhota hona hamesha 'zyada bharosemand' hone ka matlab nahi hota, jab tak underlying independence-assumption khud check na ki jaaye.

Dono Parts Ko Jodkar Dekho — Ek Study Jo Apne Aap Ko Do Baar Check Karti Hai

Is PDF ka poora arc ek achha example hai us tareeke ka jismein ek responsible research project apne headline results ko treat karta hai: sirf ek significant p-value mil jaana kaafi nahi hai. Part A ne poochha 'agar hum ek saath 8 tests chala rahe hain, toh kya humein apna threshold sakht karna chahiye?' — jawaab haan tha, aur us sakht threshold ke baad bhi do H3 findings bach gaye. Part B ne ek bilkul alag, doosra sawaal poochha — 'kya humare 251 observations sach mein independent hain?' — jawaab tha 'poori tarah nahi,' aur us second correction ke baad bhi dono findings bach gaye, lekin ek jaisi confidence ke saath nahi.

Development Log ki Entry 16 (Holm ki 'blanket description overreach') aur Entry 47 (Holm-vs-spatial 'sequential nahi, parallel') dono ek hi underlying discipline dikhati hain — jab bhi paper ki prose kisi complex statistical result ko oversimplify karti hai, chahe wo galti direction mein hi ho (sabko 1.000 keh dena, ya do independent tests ko chained batana), asli code padhkar us prose ko theek kiya jaata hai. Ye is poore 22-PDF series ka ek central theme hai — jo kuch published hai, uspar blindly bharosa nahi karna, balki code aur numbers se seedha verify karna, chahe result khud already 'good news' (significant finding) ho.

RECAP — PDF 16 Mein Kya Seekha: (1) Multiple testing problem — jitne zyada tests, utna zyada chance ek galat 'significant' mil jaane ka, sirf luck se. (2) Holm-Bonferroni step-down procedure — sort ascending, rank-based multiplier, monotonicity enforcement, 1.0 par cap — poora arithmetic haath se verify kiya, 8 real p-values ke against. (3) Moran's I — ek statistic jo batata hai paas-paas ke observations kitne milte-julte hain, geography ki wajah se. (4) SAR(1) calibration aur simulation — ek 'sirf-geography, bina-real-effect' wala null distribution banane ka tareeka, jab closed-form formula available na ho. (5) Holm aur spatial corrections parallel hain, sequential nahi — dono ek hi naive result ke, do independent checks. (6) Dono H3 findings — NDBI-proximity aur lights-proximity — Holm aur spatial, dono corrections ke baad bhi zinda hain, lekin lights-proximity zyada comfortably, NDBI-proximity sirf-thoda-sa.

Aage, [PDF](#) [17](#) mein [extended_robustness_checks.py](#), [geocoding_selection_bias_check.py](#), aur [power_analysis.py](#) dekhenge — teen aur alag angles se study ki robustness ko test karne wale scripts, jo poori tarah alag sawaal poochhte hain: kya geocoding mein koi systematic bias hai, aur kya is study ka sample size chhote-lekin-real effects dhoondhne ke liye kaafi bada hai.